

MORKQL

A METTA-Shaped Query Language for MORK Path Spaces

Draft Language and Compilation Specification, Version 0.1

Proposal for discussion and prototyping

July 12, 2026

Abstract

MORKQL is a declarative, S-expression query language intended for end-user access to MORK knowledge bases and for direct embedding in METTA. It presents familiar pattern queries, set composition, and Datalog-style recursive rules while compiling without a new runtime layer to the existing MORKL algebra of finite path spaces. The language is designed so that elaboration can produce a checkable certificate relating a surface query, its logical plan, its physical path plan, and the emitted MORKL program. This document specifies the proposed syntax, static semantics, denotational semantics, lowering rules, verification profile, METTA API, and a lossless low-level escape hatch called MORKQL Core.

Status. This is a design proposal, not an official standard of the MORK, MORKL, ZIPPY, METTA, or OpenCog Hyperon projects. The specification deliberately separates stable semantic requirements from implementation-dependent details such as the current byte encoding of MORK expressions and the exact serialized form of proof certificates.

Contents

1	Introduction	5
1.1	Purpose	5
1.2	Scope	5
1.3	Normative language	6
1.4	ASCII source requirement	6
2	Background	6
2.1	MORK data and path tries	6
2.2	MORKL and Zippy	6
2.3	MeTTa as a host syntax	6
2.4	The usability gap	7
3	Goals, Non-Goals, and Design Decisions	7
3.1	Primary goals	7
3.2	Non-goals	7
3.3	Why <code>find</code> is the central form	7
3.4	Why Datalog-shaped rules are used for recursion	7
3.5	Why queries are quoted data	8

3.6	Why set semantics are mandatory	8
3.7	Why logical relations are separated from physical layouts	8
3.8	Why pure queries and updates are separate	8
3.9	Why there is a core language	8
4	Semantic Domains	8
4.1	Paths and spaces	8
4.2	Logical atoms	9
4.3	Variables and substitutions	9
4.4	Knowledge sources	9
4.5	Pattern matching	9
5	Surface Syntax	9
5.1	Lexical conventions	9
5.2	Abstract grammar	10
5.3	The <code>find</code> form	10
5.4	Goals	11
5.5	Set queries	11
5.6	Rule queries	11
5.7	Core queries	11
6	Schemas, Relations, and Physical Layouts	11
6.1	Schema declaration	11
6.2	Open and strict schema modes	12
6.3	Layouts	12
6.4	Index coherence	12
6.5	Binding modes and index choice	13
7	Static Semantics	13
7.1	Compilation environments	13
7.2	Variable scope	13
7.3	Range restriction	13
7.4	Negation safety	13
7.5	Grounded tests	14
7.6	Type checking	14
7.7	Rule dependencies	14
7.8	Monotonicity of recursive components	14
7.9	Finite-range recursion	14
7.10	Static errors	14
8	Denotational Semantics	15
8.1	Goal satisfaction	15
8.2	Find	15
8.3	Long query form	16
8.4	Set forms	16
8.5	Rule groups	16
8.6	Result order and materialization	16

9	Compilation to MORKL	16
9.1	Required stages	16
9.2	The MQT macro	17
9.3	Positive find lowering	17
9.4	Equality lowering	17
9.5	Inequality and grounded tests	17
9.6	Safe negation lowering	18
9.7	Set-query lowering	18
9.8	Rule lowering	18
9.9	Join reordering	18
9.10	Physical index selection	19
9.11	Worked lowering: mother query	19
10	MORKQL Core	19
10.1	Purpose	19
10.2	Core path expressions	19
10.3	Core space expressions	20
10.4	Core binding scope	20
10.5	Core example	21
10.6	Target compatibility	21
11	MeTTa Integration API	21
11.1	Portable quoted form	21
11.2	Result handles	21
11.3	Compilation and inspection	22
11.4	Named views	22
11.5	Updates	22
12	Verification and Proof-Carrying Compilation	22
12.1	Verification objective	22
12.2	Why the surface language helps verification	23
12.3	Required proof obligations	23
12.4	Certificate shape	23
12.5	Certificate checker	24
12.6	Trusted computing base	24
12.7	Connection to the Zippy verification stack	24
12.8	Verification profiles	25
13	Examples	25
13.1	Family data	25
13.2	Simple selection	25
13.3	A two-pattern join	25
13.4	Long query form	26
13.5	Repeated-variable constraint	26
13.6	Nested atoms	26
13.7	Safe negation	26
13.8	Grounded test	27
13.9	Transitive closure	27

13.10A helper relation and exports	27
13.11Schema-guided index selection	28
13.12Inspecting emitted MORKL	28
13.13Proof-carrying compilation	29
13.14Pure query followed by explicit update	29
14 Conformance	29
14.1 Base implementation requirements	29
14.2 Verified implementation requirements	29
14.3 Determinism	30
14.4 Forward compatibility	30
15 Open Design Questions	30
A Compact Grammar Reference	30
B Compiler Correctness Skeleton	31
C Recommended Explain-Plan Schema	32
D Glossary	33

1 Introduction

1.1 Purpose

MORKL provides a compact algebra for finite sets of structured paths. Its constructors are well suited to formal reasoning and low-level optimization, but ordinary users usually think in terms of facts, variables, conjunctions, derived relations, graph reachability, and named views. MORKQL bridges those levels.

A MORKQL implementation accepts a quoted METTA-shaped query, checks it, chooses physical path layouts, and immediately emits ordinary MORKL. The backend therefore need not implement a separate query evaluator. Existing MORKL optimizers, trie executors, zipper executors, and proof machinery remain the execution substrate.

The intended compilation chain is:

```

MeTTa atom containing MORKQL
  |
  v
parse and macro-expand
  |
  v
checked logical query IR
  |
  v
binding-mode and index planning
  |
  v
MORKQL Core / initial MORKL
  |
  v
optimized MORKL
  |
  v
trie, zipper, or operation-graph execution

```

1.2 Scope

This document specifies:

- a surface query language based on **find**, set operators, and Datalog-shaped **rules**;
- schema declarations that separate logical relations from physical path layouts and indexes;
- safety, typing, recursion, negation, and grounded-operation checks;
- a denotational semantics over finite sets of ground atoms and finite path spaces;
- lowering to MORKL, including the use of the multi-pattern-query macro **MQT** and union-saturating fixed points;
- a one-to-one low-level syntax for MORKL constructors;
- a METTA-facing API and a proof-carrying compilation result; and
- conformance requirements and worked examples.

This document does not standardize transactions, persistence, distribution, a cost model, the internal MORK byte encoding, or the complete METTA evaluator.

1.3 Normative language

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** indicate requirements on a conforming implementation. Examples and paragraphs introduced by “Design rationale” are informative unless a normative rule says otherwise.

1.4 ASCII source requirement

This specification source is restricted to seven-bit ASCII. Mathematical symbols are written as LaTeX commands, and all example programs use ASCII tokens. A conforming parser may support Unicode aliases, but the ASCII spelling in this document is canonical.

2 Background

2.1 MORK data and path tries

MORK stores expressions in a prefix-oriented representation backed by a radix trie. The current MORK documentation describes S-expressions as sequences containing arity information, symbols, new-variable markers, and variable references. It also emphasizes a practical consequence: queries are most efficiently indexed when a large ground prefix precedes unbound material [5].

The concrete byte encoding is explicitly subject to change. MORKQL therefore specifies an abstract, injective encoding function from logical atoms to paths rather than freezing a particular byte layout.

2.2 MORKL and Zippy

The ZIPPY formalization presents a space as a finite set of words over path items. The same value can be interpreted as a relation, a tagged database, a trie, a finite language, or a graph encoding. The core includes union, intersection, subtraction, path composition, prefix restriction, wrapping, unwrapping, tails, first-symbol iteration, grounded operations, and recursion [2, 3].

The key structural operator is MORKL *Iteration*. It groups a path space by its first item, binds the head item, binds the set of remaining tails, and evaluates a template for each group. In the reference semantics, joins and projections can be implemented by composing iteration with prefix operations. Recursive Datalog examples use union-saturating fixed points [2].

The formalization also supplies set, eager-trie, zipper, and operation-graph views, with proof obligations connecting optimized programs to the reference path-set semantics [1, 4].

2.3 MeTTa as a host syntax

METTA represents programs and data using atoms and expressions, supports variables with a dollar-sign spelling, and exposes atom spaces through grounded operations. The Hyperon implementation and documentation make quoted atoms and custom spaces a natural place to host a query DSL [7, 8].

MORKQL uses METTA-shaped syntax but defines its own closed query AST. A quoted MORKQL expression is data for the MORKQL compiler; it is not arbitrary METTA code to be reduced by the general evaluator.

2.4 The usability gap

A low-level algebraic term exposes implementation-relevant structure but makes simple questions verbose. For example, a user asking for mothers wants to state two facts sharing a variable:

```
(find (Mother $child $mother)
      (child $child $mother)
      (female $mother))
```

The corresponding MORKL plan may involve nested pattern unification, restriction, unwrapping, intersection, iteration, and wrapping. Those operations should remain visible in an explain plan and available in a core escape hatch, but they should not be mandatory surface syntax.

3 Goals, Non-Goals, and Design Decisions

3.1 Primary goals

1. **Immediate lowering.** Every accepted surface construct expands to ordinary MORKL before execution. No new backend query operator is required.
2. **MeTTa-shaped use.** Queries are ordinary S-expressions that can be quoted, stored, generated, transformed, and passed through METTA.
3. **Declarative semantics.** The meaning of a query is independent of join order, index choice, and materialization strategy.
4. **Trie-aware planning.** The compiler may exploit grounded prefixes, alternate path layouts, and virtual zipper views.
5. **Verification support.** Static checks and compilation steps expose small proof obligations and explicit trust boundaries.
6. **Lossless escape hatch.** Advanced users can write every supported MORKL constructor through MORKQL Core.

3.2 Non-goals

MORKQL is not intended to replace general METTA reduction, define a universal ontology, hide all physical costs, guarantee termination of arbitrary grounded code, or standardize distributed transactions. It also does not require SQL-like bags, nulls, implicit coercions, or global result ordering.

3.3 Why find is the central form

Design rationale. A multi-pattern comprehension is the smallest friendly abstraction that covers selection, projection, joins, and result construction. The existing MORKL examples already define a multi-pattern-query helper, `MQT`, that recursively unifies a list of path patterns and emits a template when the bindings are consistent. Giving that pattern a first-class surface spelling adds ergonomics without adding a new semantic primitive.

3.4 Why Datalog-shaped rules are used for recursion

Design rationale. Users recognize reachability and derived relations more readily as rules than as a hand-written recursive algebra term. Positive rule groups have a direct least-fixed-point meaning, admit standard safety and stratification checks, and compile to MORKL `Fixpoint` plus unions of `MQT` expansions.

3.5 Why queries are quoted data

Design rationale. Treating a query as a closed AST prevents accidental evaluation by unrelated METTA rules, makes alpha-renaming and hashing deterministic, and gives the compiler a finite language to type-check and certify. An implementation may register a non-eager grounded operation that removes the need for explicit `quote`, but quoted syntax is the portable form.

3.6 Why set semantics are mandatory

Design rationale. MORKL spaces are finite sets of paths. Making MORKQL set-valued preserves direct lowering, gives idempotent union, avoids duplicate-sensitive proof obligations, and matches the semantics of graph reachability and Datalog consequence sets. Users who need multiplicity must encode it explicitly as data.

3.7 Why logical relations are separated from physical layouts

Design rationale. In a prefix trie, argument order can dominate lookup cost. A logical predicate such as `parent(parent, child)` should not change meaning merely because the engine chooses a child-first index. Schema layouts allow the planner to select a grounded prefix while a certificate records the index-coherence fact on which that choice depends.

3.8 Why pure queries and updates are separate

Design rationale. A pure query denotes a space. An update denotes a state transition. Keeping the two forms distinct simplifies composition, caching, equivalence checking, and proofs. All mutating API operations therefore carry an exclamation mark.

3.9 Why there is a core language

Design rationale. Not every path-space program is naturally relational. Prefix cylinders, universal tail intersection, folds, and explicit virtual tries are valuable low-level tools. A lossless core syntax prevents the friendly surface from becoming an expressive bottleneck and supplies a readable target for compiler output.

4 Semantic Domains

4.1 Paths and spaces

Let Σ be the set of MORK path items. A path is a finite word over Σ :

$$\text{Path} = \Sigma^*.$$

The empty path is ϵ . A path space is a finite set of paths:

$$\text{Space} = \mathcal{P}_{\text{fin}}(\text{Path}).$$

Concatenation of paths is written $p \cdot q$. Concatenation of spaces is language product:

$$A \cdot B = \{p \cdot q \mid p \in A, q \in B\}.$$

4.2 Logical atoms

A logical atom is a ground METTA atom accepted by the configured MORK codec. Let Atom_g denote the set of such ground atoms. Variables are not members of Atom_g ; they occur only in templates and patterns.

A MORK codec supplies partial inverse functions:

$$\text{Enc} : \text{Atom}_g \rightarrow \text{Path}, \quad \text{Dec} : \text{Path} \rightarrow \text{Atom}_g,$$

with the required round-trip property:

$$\text{Dec}(\text{Enc}(a)) = a.$$

An implementation MAY use relation-specific layouts instead of the default codec. Each selected layout must be injective over the relation's well-typed ground facts.

4.3 Variables and substitutions

A variable token begins with $\$$. A substitution $\theta \in \text{Subst}$ is a finite map from variables to ground terms. Applying θ to a term, atom, pattern, or template is written $\theta(x)$.

The token $\$_{_}$ is anonymous. Each occurrence is alpha-renamed to a distinct fresh variable before any other analysis.

4.4 Knowledge sources

A logical source D is a finite set of ground atoms. A physical source is a MORK space containing one or more coherent layouts of those facts. Query semantics is defined over D ; compilation and execution operate over physical path spaces.

4.5 Pattern matching

For a pattern P and ground atom a , the relation

$$\text{match}(P, a) = \theta$$

holds when first-order structural unification succeeds and returns substitution θ . Repeated occurrences of a variable must receive the same ground term. Pattern matching does not execute arbitrary METTA reduction.

5 Surface Syntax

5.1 Lexical conventions

MORKQL reuses the host METTA tokenizer. Symbols, strings, numbers, variables, and parenthesized expressions are host atoms. A semicolon begins a line comment where the host parser permits comments.

The following tokens are reserved in query position:

```
find query where yield union intersect except rules export
<- not test core
```

A relation may use one of those symbols only through an implementation-defined escaping mechanism.

5.2 Abstract grammar

The following grammar is informative EBNF. The host parser first produces atoms; the MORKQL parser then recognizes the forms below.

```

query          ::= find-query
                | long-query
                | set-query
                | rules-query
                | core-query

find-query     ::= "(" "find" template goal* ")"

long-query     ::= "(" "query"
                  "(" "where" goal* ")"
                  "(" "yield" template ")"
                  ")"

set-query      ::= "(" "union" query query+ ")"
                | "(" "intersect" query query+ ")"
                | "(" "except" query query ")"

rules-query    ::= "(" "rules" rule-option* clause+ ")"
rule-option    ::= "(" "export" relation-name+ ")"
clause        ::= "(" "<-" relation-pattern goal* ")"

core-query     ::= "(" "core" core-space ")"

goal           ::= relation-pattern
                | "(" "not" relation-pattern ")"
                | "(" "=" term term ")"
                | "(" "!=" term term ")"
                | "(" "test" grounded-name term* ")"

template      ::= term
relation-pattern ::= expression-with-symbol-head
term          ::= host-atom

```

5.3 The find form

```
(find TEMPLATE GOAL-1 ... GOAL-N)
```

The form emits one instantiated copy of `TEMPLATE` for every consistent substitution satisfying all goals. Results are deduplicated by set semantics.

The long form is exact sugar:

```
(query
  (where GOAL-1 ... GOAL-N)
  (yield TEMPLATE))
```

expands to:

```
(find TEMPLATE GOAL-1 ... GOAL-N)
```

5.4 Goals

A relation pattern is matched against facts in the current logical source.

(= A B) adds a unification constraint. It may bind an otherwise unbound variable when doing so is unambiguous and type-correct.

(!= A B) succeeds when both sides are ground under the current substitution and are not structurally equal.

(not P) succeeds when the variables of P are already determined by the positive part of the query and no matching fact exists.

(test F A ...) invokes a declared, pure grounded predicate. All arguments must be bound before the test is scheduled.

5.5 Set queries

union, intersect, and except combine the result spaces of subqueries. They use ordinary set union, intersection, and relative complement.

```
(union
  (find (Person $x) (female $x))
  (find (Person $x) (male $x)))
```

5.6 Rule queries

A rule query contains one or more clauses:

```
(rules
  (export path)

  (<- (path $x $y)
      (edge $x $y))

  (<- (path $x $z)
      (path $x $y)
      (edge $y $z)))
```

The head of each clause is a relation pattern. The body is a conjunction of goals. By default every relation appearing in a rule head is exported. An **export** option restricts the result to the listed head relations while still permitting unexported helper relations inside the fixed point.

5.7 Core queries

(core C) asserts that C is a MORKQL Core space expression. Core syntax is defined in Section 10. Core programs bypass relational planning but still undergo sorting, scope, and target-version checks.

6 Schemas, Relations, and Physical Layouts

6.1 Schema declaration

A recommended schema form is:

```
(morkql:schema family-v1
  (type Person)

  (relation child
    (fields
      (child Person)
      (parent Person))

    (layout canonical
      (child $child $parent))

    (index by-parent
      (child-by-parent $parent $child)
      (maintained true)))

  (relation female
    (fields
      (person Person))

    (layout canonical
      (female $person))))
```

This declaration is host-level configuration, not a query. It declares a logical relation, named fields, field types, a canonical path template, and optional secondary indexes.

6.2 Open and strict schema modes

In *open mode*, an undeclared relation uses the default MORK codec for the whole atom. Arity and domain types are inferred only from local use.

In *strict mode*, every relation in a query, rule head, or output template must be declared. Strict mode is required by the Verified MORKQL profile.

6.3 Layouts

For relation R with fields $f_1, \dots, f_n$, a layout L is an injective path template containing every field exactly once, possibly in a different order and possibly under a different physical tag.

For example, the logical fact

```
(child Bob Pam)
```

may have layouts:

```
canonical:  child.Bob.Pam
by-parent:  child-by-parent.Pam.Bob
```

The dot notation is explanatory; the actual codec may use structural path items.

6.4 Index coherence

A secondary index is coherent when decoding every indexed path through its declared inverse produces exactly the same logical fact set as the canonical layout.

A conforming implementation must classify each index as one of:

- **maintained**: generated or transactionally maintained from the canonical relation by the MORKQL runtime;
- **proved**: accompanied by a checked derivation of coherence;
- **assumed**: supplied externally and listed as a certificate assumption; or
- **untrusted**: usable only outside the Verified profile.

6.5 Binding modes and index choice

For every candidate plan, the compiler computes which variables are bound before a relation lookup. It SHOULD select a layout whose longest prefix is ground under those bindings. The choice affects cost but not denotation.

An explain plan must show both the logical relation and selected physical layout.

7 Static Semantics

7.1 Compilation environments

Static checking uses:

- a relation environment Γ_R mapping relation names to field types and layouts;
- a grounded environment Γ_G mapping grounded names to signatures and declared properties;
- a variable environment Γ_V mapping variables to inferred types; and
- a source environment identifying the default physical space and schema.

7.2 Variable scope

Variables are scoped to the nearest **find** or rule clause. Variables with the same spelling in different clauses are distinct. Rule-head variables and rule-body variables share one clause scope.

The compiler must alpha-normalize variables before planning. Generated variables must not collide with user variables.

7.3 Range restriction

A **find** query is safe only if every variable in the output template occurs in a positive relation goal or is determined by a chain of equality constraints from such a variable or a ground constant.

A rule is safe only if every variable in its head occurs in a positive body relation goal or is similarly determined.

For example, the following query is rejected because **\$y** is unconstrained:

```
(find (Pair $x $y)
      (person $x))
```

7.4 Negation safety

Every variable in **(not P)** must be bound by positive goals or equalities in the same query or clause. Negation never introduces bindings.

The compiler may reorder conjunctions, so “already bound” is a dependency condition rather than a source-order condition.

7.5 Grounded tests

Every argument to `test` and `!=` must be ground when the goal executes. A grounded predicate declaration has the shape:

```
(morkql:grounded less-than
  (type (-> Number Number Bool))
  (properties pure total deterministic))
```

A test used by a Verified query must be declared at least `pure`, `total`, and `deterministic`. The checker may require an implementation hash or proof reference.

7.6 Type checking

For a declared relation, each argument is checked against the corresponding field type. Repeated variables impose type equality. A template is checked structurally; strict mode requires its outer relation to be declared.

Implementations may support subtype or coercion systems, but any coercion used in a Verified query must be explicit in the certificate.

7.7 Rule dependencies

A rule dependency graph has one node per head relation. A positive body reference adds a positive edge; a negated body reference adds a negative edge.

A rule group is stratified when no strongly connected component contains a negative edge. Strata are evaluated in dependency order. Relations from lower strata are constant inputs to higher strata.

7.8 Monotonicity of recursive components

A recursive strongly connected component is accepted in the Verified profile only when its immediate-consequence operator is monotone. Positive relation goals, unification, inequality over already-bound values, and pure value tests are monotone. Negation of a relation in the same component, subtraction depending on the current recursive value, and aggregates over the current recursive value are not monotone.

7.9 Finite-range recursion

Monotonicity alone does not guarantee termination if rules can construct an unbounded sequence of fresh terms. The Verified profile therefore requires a finite-range proof. The standard sufficient condition is function-free Datalog: recursive heads may contain relation structure, variables from positive body goals, and ground constants, but may not wrap a recursively varying term in a fresh constructor.

A grounded operation used in recursion must additionally be declared `finite-range`, or the compiler must derive an equivalent finite-domain bound.

7.10 Static errors

A compiler should report structured errors. Recommended error codes include:

Code	Meaning
<code>unknown-relation</code>	A strict-schema query refers to an undeclared relation.
<code>arity-mismatch</code>	A relation pattern has the wrong number of fields.
<code>type-mismatch</code>	A term cannot have the required field or result type.
<code>unsafe-output-variable</code>	A template or rule-head variable is not range restricted.
<code>unsafe-negation-variable</code>	A negated pattern contains an unbound variable.
<code>unbound-test-argument</code>	A test or inequality can execute before its inputs are ground.
<code>unstratified-negation</code>	A negative dependency occurs inside a recursive component.
<code>nonmonotone-recursion</code>	A recursive body is not monotone in its recursive relations.
<code>unproved-finite-range</code>	Termination cannot be established for verified recursion.
<code>untrusted-grounded-op</code>	A grounded operation lacks the properties required by the profile.
<code>incoherent-index</code>	A selected physical layout is known to disagree with canonical data.
<code>unsupported-morkl-target</code>	Core syntax requires a constructor absent from the target version.

8 Denotational Semantics

8.1 Goal satisfaction

Let D be the current logical source and I the current derived-relation set for a rule component. Let $K = D \cup I$.

For a positive relation pattern P :

$$K, \theta \models P \quad \text{iff} \quad \theta(P) \in K.$$

For equality:

$$K, \theta \models (= \ t_1 \ t_2) \quad \text{iff} \quad \theta(t_1) = \theta(t_2),$$

where unification may extend θ before both sides are ground.

For inequality:

$$K, \theta \models (! = \ t_1 \ t_2) \quad \text{iff} \quad \theta(t_1), \theta(t_2) \text{ are ground and unequal.}$$

For safe negation:

$$K, \theta \models (\text{not } P) \quad \text{iff} \quad \theta(P) \notin K.$$

A grounded test is satisfied exactly when its declared predicate returns true on the ground arguments.

8.2 Find

For a template T and goals $G_1, \dots, G_n$:

$$\llbracket (\text{find } T \ G_1 \cdots G_n) \rrbracket_D = \{\text{Enc}(\theta(T)) \mid \theta \text{ is safe and } D, \theta \models G_i \text{ for every } i\}.$$

The result is a path space. At the logical level it may equivalently be viewed as the set of ground atoms $\theta(T)$.

With no goals, the template must be ground and the result is the singleton $\{\text{Enc}(T)\}$.

8.3 Long query form

$$\llbracket (\text{query } (\text{where } \vec{G}) \text{ (yield } T)) \rrbracket_D = \llbracket (\text{find } T \vec{G}) \rrbracket_D.$$

8.4 Set forms

$$\begin{aligned} \llbracket (\text{union } Q_1 \cdots Q_n) \rrbracket_D &= \bigcup_i \llbracket Q_i \rrbracket_D, \\ \llbracket (\text{intersect } Q_1 \cdots Q_n) \rrbracket_D &= \bigcap_i \llbracket Q_i \rrbracket_D, \\ \llbracket (\text{except } Q_1 \ Q_2) \rrbracket_D &= \llbracket Q_1 \rrbracket_D \setminus \llbracket Q_2 \rrbracket_D. \end{aligned}$$

8.5 Rule groups

Let P be a stratified rule program. For one positive stratum, let H be the set of head relation names and let

$$I_0 = \{a \in D \mid \text{rel}(a) \in H\}.$$

The immediate-consequence operator is:

$$T_{P,D}(I) = \{\theta(H_r) \mid (H_r \leftarrow G_1, \dots, G_n) \in P, D \cup I, \theta \models G_j \text{ for every } j\}.$$

The stratum meaning is the least fixed point:

$$I^* = \text{lfp } I. I_0 \cup T_{P,D}(I).$$

If an **export** option lists relations E , the query result is:

$$\{\text{Enc}(a) \mid a \in I^*, \text{rel}(a) \in E\}.$$

Without **export**, $E = H$. Stratified programs evaluate each stratum with lower strata added to D .

8.6 Result order and materialization

A query denotes a set and has no semantic iteration order. A consumer may expose the canonical MORK path order for reproducibility, but order is not part of ordinary query equivalence.

A query may be represented as a virtual space. Denotation does not require eager materialization.

9 Compilation to MORKL

9.1 Required stages

A conforming compiler performs the following conceptual stages:

1. parse the quoted atom as a MORKQL AST;
2. expand surface macros, including the long **query** form;

3. alpha-normalize variables and replace every $\$_$ with a fresh variable;
4. resolve schemas, relation fields, and grounded declarations;
5. check types, safety, stratification, monotonicity, and finite range;
6. construct a logical binding plan;
7. choose physical layouts and a legal goal order;
8. lower the logical plan to MORKQL Core or directly to MORKL;
9. optionally apply certified MORKL rewrites; and
10. emit code, explain information, and optionally a proof certificate.

No surface-only constructor may remain in the executable backend term.

9.2 The MQT macro

The ZIPPY examples define a multi-pattern query helper of the schematic form:

```
MQT(source, [P1, ..., Pn], template)
```

It successively matches each path pattern against **source**, carries a consistent binding environment, and emits the instantiated template after the last pattern. MORKQL adopts MQT as a compiler macro, not as a new MORKL constructor.

The required semantic theorem is:

$$\llbracket \text{MQT}(S, [P_1, \dots, P_n], T) \rrbracket = \{\theta(T) \mid \theta(P_i) \in \llbracket S \rrbracket \text{ for all } i\}.$$

A compiler may expand MQT using nested iteration, restriction, unwrapping, intersection, wrapping, and singleton construction, or call an existing trusted macro expander whose result is ordinary MORKL.

9.3 Positive find lowering

After layout selection, every relation goal is a physical path pattern. A positive query lowers schematically as:

$$\text{compile}_D(\text{find } T \ P_1 \cdots P_n) = \text{MQT}(D, [L_1(P_1), \dots, L_n(P_n)], L_T(T)),$$

where each L_i is the selected coherent layout and L_T is the output layout or default codec.

A ground, goal-free **find** lowers to MORKL **Singleton**.

9.4 Equality lowering

Variable-variable equality is normalized by union-find over compiler variables. Variable-ground equality becomes a binding or a constant constraint. Equality that remains after normalization is emitted as an explicit grounded structural test.

This normalization step is semantics preserving when substitutions are rewritten consistently throughout all patterns and the template.

9.5 Inequality and grounded tests

A grounded Boolean predicate lowers to an explicit grounded MORKL boundary. One valid implementation encodes its bound arguments as a path and uses a path-to-space grounded operation that returns $\{\epsilon\}$ for true and $\emptyset$ for false. The returned gate is composed with the current binding row.

The certificate must identify the grounded function and the properties assumed or proved about it.

9.6 Safe negation lowering

Negation is compiled as an anti-join over an internal binding space.

Let B be the space of encoded substitutions produced by positive goals. Let B_N be the subset of those substitutions for which the negated pattern has a match. Then:

$$B_{ok} = B \setminus B_N.$$

The output template is instantiated only from B_{ok} . This transformation uses MORKL subtraction but does not expose binding-row encoding to the user.

Because every variable in the negated pattern is already represented in B , no new binding can be introduced by the anti-join.

9.7 Set-query lowering

MORKQL	MORKL target
(union Q1 ... Qn)	A balanced tree of Union constructors.
(intersect Q1 ... Qn)	A balanced tree of Intersection constructors.
(except Q1 Q2)	<code>Subtraction(compile(Q1), compile(Q2))</code> .

Balancing is an optimization and does not affect denotation.

9.8 Rule lowering

For each clause $H_i \leftarrow B_i$, compile one consequence branch:

$$C_i(R) = \text{MQT}(D \cup R, \text{patterns}(B_i), H_i),$$

with tests, equalities, and safe negation lowered as described above. Let

$$C(R) = \bigcup_i C_i(R).$$

The positive stratum lowers schematically to:

$$\text{Fixpoint}(I_0, \text{rec}, \text{rec} \cup C(\text{rec})).$$

The explicit union with **rec** makes the next iterate inflationary. The result is then restricted to exported head layouts. A backend may use a semi-naive plan so long as it proves or otherwise certifies equivalence to the same least fixed point.

9.9 Join reordering

Positive relation goals form a logical conjunction. The planner may permute them to increase grounded-prefix length or reduce intermediate binding spaces. The compiler must respect dependencies of equality, negation, inequality, and grounded tests.

A proof certificate for a positive reordering need only record a permutation and instantiate generic commutativity and associativity lemmas for conjunction or the corresponding MQT theorem.

9.10 Physical index selection

Selecting a secondary layout replaces a logical pattern by the declared physical path template. The compiler records:

- the logical relation and field substitution;
- the chosen layout and argument permutation;
- the index-coherence evidence class; and
- the grounded-prefix estimate used for planning.

The result template is always decoded to the logical output shape before being encoded under its declared output layout.

9.11 Worked lowering: mother query

Surface query:

```
(find (Mother $person $mother)
      (child $person $mother)
      (female $mother))
```

With canonical flat explanatory layouts, the initial target is:

```
MQT(
  family,
  [ child.$person.$mother,
    female.$mother ],
  Mother.$person.$mother)
```

If `$person` is ground at call time and `child` has a child-first layout, the planner may use it directly. If `$mother` is ground, it may instead use a parent-first secondary index. Both plans share the same logical proof obligation.

10 MORKQL Core

10.1 Purpose

MORKQL Core is a readable, lossless S-expression spelling of the MORKL AST. It is used for advanced programs, compiler output, debugging, round trips, and proof artifacts. Core is not intended to be the default end-user notation.

A conforming pretty-printer must satisfy alpha-equivalent round trip:

$$\text{parse}(\text{print}(C)) \equiv_{\alpha} C.$$

10.2 Core path expressions

Recommended path forms are:

```
(path-ref NAME)
(path-lit ITEM-1 ... ITEM-N)
(path-cat PATH-1 ... PATH-N)
(path-ground-pp GROUNDED-NAME PATH)
(path-ground-sp GROUNDED-NAME SPACE)
```

They map respectively to MORKL `Path.Deref`, `Path.Constant`, nested `Path.Concat`, `Path.GroundedPP`, and `Path.GroundedSP`.

10.3 Core space expressions

MORKQL Core	MORKL constructor
<code>(empty)</code>	<code>Empty</code>
<code>(space NAME)</code>	<code>Mention(NAME)</code>
<code>(one PATH)</code>	<code>Singleton(PATH)</code>
<code>(literal PATH-1 ... PATH-N)</code>	Literal containing the listed constant paths
<code>(union A B ...)</code>	<code>Nested Union</code>
<code>(intersect A B ...)</code>	<code>Nested Intersection</code>
<code>(except A B)</code>	<code>Subtraction(A,B)</code>
<code>(under DATA PREFIXES)</code>	<code>Restriction(DATA,PREFIXES)</code>
<code>(outside DATA PREFIXES)</code>	<code>Raffination(DATA,PREFIXES)</code>
<code>(concat-spaces A B)</code>	<code>Composition(A,B)</code>
<code>(each-head SRC HEAD-NAME TAILS-NAME BODY)</code>	<code>Iteration(SRC,HEAD-NAME,TAILS-NAME,BODY)</code>
<code>(fix INIT REC-NAME BODY)</code>	<code>Fixpoint(INIT,REC-NAME,BODY)</code>
<code>(fold-heads SRC INIT ACC HEAD TAILS OUT UPDATE)</code>	<code>Fold(SRC,INIT,ACC,HEAD,TAILS,OUT,UPDATE)</code>
<code>(add-prefix PATH SRC)</code>	<code>Wrap(SRC,PATH)</code>
<code>(remove-prefix SRC PATH)</code>	<code>Unwrap(SRC,PATH)</code>
<code>(tails SRC)</code>	<code>TailsUnion(SRC)</code>
<code>(common-tails SRC)</code>	<code>TailsIntersection(SRC)</code>
<code>(ground-ps NAME PATH)</code>	<code>GroundedPS(PATH,NAME)</code>
<code>(ground-ss NAME SRC)</code>	<code>GroundedSS(SRC,NAME)</code>
<code>(range SRC LO HI)</code>	<code>Range(SRC,LO,HI)</code>
<code>(call NAME (PATHS ...) (SPACES ...))</code>	<code>Call(NAME,PATHS,SPACES)</code>

The ordering of arguments in the syntax is chosen for readability. The mapping in the right column is normative.

10.4 Core binding scope

In:

```
(each-head SOURCE head tails BODY)
```

head names a path reference and **tails** names a space reference in **BODY**. In:

```
(fix INIT rec BODY)
```

rec names a space reference in **BODY**. Core names are lexically scoped and alpha-renamed during compilation.

10.5 Core example

The query “all facts under either the `female` or `male` prefix” can be written schematically as:

```
(core
  (under
    (space family)
    (literal
      (path-lit female)
      (path-lit male)))))
```

Under a structural MORK expression codec, a schema-aware compiler would generally emit different physical path literals. The example illustrates the algebraic shape, not a mandatory byte encoding.

10.6 Target compatibility

A Core program declares or inherits a MORKL target version. A compiler must reject a constructor not supported by that target rather than silently changing semantics. Derived aliases may be expanded when an equivalence theorem is available.

11 MeTTa Integration API

11.1 Portable quoted form

The portable invocation is:

```
!(morkql:q &source
  (quote QUERY))
```

A host implementation may make `morkql:q` non-eager and permit:

```
!(morkql:q &source QUERY)
```

but it must preserve the same query AST and must not reduce `QUERY` as arbitrary MeTTa code before MORKQL compilation.

11.2 Result handles

`morkql:q` returns a `MorkSpace` handle. The handle may refer to a stored space, an eagerly materialized result, or a virtual MORKL/zipper view.

Recommended consumers are:

```
!(morkql:collect &result)
!(morkql:first &result)
!(morkql:exists? &result)
!(morkql:count &result)
```

`collect` converts paths back to logical atoms when a decoder is available. The order of collected results is not semantically significant.

11.3 Compilation and inspection

Recommended operations are:

```
!(morkql:explain &source (quote QUERY))
!(morkql:compile &source (quote QUERY))
!(morkql:compile/proof &source (quote QUERY))
!(morkql:check-equivalent &source
  (quote QUERY-1)
  (quote QUERY-2))
```

`explain` returns logical and physical plans. `compile` returns a Core or serialized MORKL term. `compile/proof` also returns a certificate.

`check-equivalent` has three permitted results:

```
(proved CERTIFICATE)
(disproved COUNTEREXAMPLE)
(unknown REASON)
```

It must not report `proved` solely because two queries happen to agree on one test source unless the claim is explicitly source-relative.

11.4 Named views

A virtual view may be bound through the host or through a convenience operation:

```
!(morkql:view! &reachable &graph
  (quote
    (rules
      (export path)
      (<- (path $x $y) (edge $x $y))
      (<- (path $x $z) (path $x $y) (edge $y $z)))))
```

The exclamation mark indicates that a host binding is created. The query itself remains pure.

11.5 Updates

Recommended mutation operations are:

```
!(morkql:insert! &source (quote ATOM))
!(morkql:delete! &source (quote ATOM))
!(morkql:replace! &source (quote OLD-ATOM) (quote NEW-ATOM))
```

An update implementation must maintain every `maintained` index atomically with its canonical relation, or fail the transaction.

12 Verification and Proof-Carrying Compilation

12.1 Verification objective

For source D , surface query Q , emitted MORKL term M , schema S , and explicit assumptions A , the central claim is:

$$A \vdash \llbracket Q \rrbracket_{D,S} = \llbracket M \rrbracket_{\text{path}(D,S)}.$$

The left side is the logical MORKQL semantics. The right side is the reference MORKL path-set semantics over the encoded physical source.

12.2 Why the surface language helps verification

The MORKQL frontend knows facts that are difficult to recover from an arbitrary MORKL AST:

- variable binders and alpha-equivalence classes;
- relation arity and domain types;
- which goals are positive, negative, or grounded;
- which recursive dependencies are monotone;
- which physical layouts implement the same logical relation; and
- which plan changes are merely conjunction permutations.

Capturing those facts before lowering reduces verification to reusable theorems and small instance data.

12.3 Required proof obligations

A Verified compiler must account for:

1. **Parsing and macro expansion.** The parsed AST corresponds to the quoted atom, and long-form syntax expands definitionally.
2. **Alpha normalization.** Renaming is capture avoiding and preserves substitutions.
3. **Static safety.** Output, negation, and grounded inputs are range restricted and well typed.
4. **Layout correctness.** Every selected physical layout is coherent with the logical relation, by maintenance, proof, or an explicit assumption.
5. **Find lowering.** The MQT expansion has the multi-pattern comprehension semantics.
6. **Planning.** Goal reordering preserves conjunction and respects dependencies.
7. **Negation lowering.** Binding-space subtraction implements safe anti-join.
8. **Rule lowering.** The union of compiled clauses is the immediate- consequence operator.
9. **Fixed point.** Monotonicity and finite range justify least-fixed-point iteration and termination.
10. **Optimization.** Every subsequent MORKL rewrite is law justified or otherwise checked against the reference semantics.

12.4 Certificate shape

A recommended certificate container is:

```
(CompiledMORKQL
  (language-version "0.1-draft")
  (morkl-target "zippy-b")

  (surface-hash "...")
  (schema-hash "...")
  (logical-plan-hash "...")
  (physical-plan-hash "...")
  (morkl-hash "...")

  (checks
    (well-sorted)
    (range-restricted)
    (stratified)
    (monotone-components path))
```

```

(finite-range-components path))

(steps
  (macro query-to-find)
  (alpha-normalize "...")
  (select-index child by-parent "...")
  (lower-find mgt-theorem "...")
  (lower-fixpoint datalog-lfp-theorem "...")
  (optimize "law-instance-list"))

(assumptions
  (index-coherent child by-parent "maintenance-proof-id")
  (grounded less-than "implementation-hash" pure total deterministic))

(claim
  (= (denote-surface "surface-hash")
     (denote-morkl "morkl-hash"))))

```

The concrete serialization may differ, but the information must be recoverable.

12.5 Certificate checker

The checker should be much smaller than the optimizing compiler. It need not repeat cost estimation. It replays structural translations, checks theorem instances, validates hashes and scoping, and verifies every assumption against the configured trust policy.

12.6 Trusted computing base

A deployment should state its trusted computing base. At minimum it contains:

- the parser or a parser-to-AST validation step;
- the MORK atom/path codec and layout decoders;
- the MORKL reference semantics or its formal counterpart;
- the certificate checker; and
- grounded functions listed as assumptions rather than proved code.

The optimizer, cost model, and query planner need not be trusted when all their transformations are checked.

12.7 Connection to the Zippy verification stack

The frontend certificate ends at an initial MORKL term. Existing Zippy-style proof artifacts can then connect that term to an optimized term, zipper program, and operation graph. The intended composed statement is:

```

MORKQL surface
== checked logical IR
== initial MORKL
== optimized MORKL
== zipper / operation graph
== reference path-set semantics

```

The equality signs denote checked semantic equivalence, not textual identity.

12.8 Verification profiles

This draft defines three profiles:

Base. Surface parsing, `find`, positive rules, set operators, and Core lowering. Static errors are required; proof emission is optional.

Verified. Strict schema, safe stratified negation, declared grounded tests, monotone finite-range recursion, coherent indexes, and a checkable compilation certificate are required.

Extended. Implementations may admit opaque grounded functions, unproved recursion, implementation-specific aggregates, or untrusted indexes, but must mark the resulting program as outside the Verified profile.

13 Examples

13.1 Family data

Assume the source `&family` contains:

```
(child Bob Pam)
(child Liz Pam)
(child Ann Bob)
(female Pam)
(female Liz)
(female Ann)
(male Bob)
```

13.2 Simple selection

```
!(morkql:collect
  (morkql:q &family
    (quote
      (find (Woman $x)
        (female $x))))))
```

The logical result set is:

```
{ (Woman Pam), (Woman Liz), (Woman Ann) }
```

The printed order may differ.

13.3 A two-pattern join

```
!(morkql:collect
  (morkql:q &family
    (quote
      (find (Mother $child $mother)
        (child $child $mother)
        (female $mother))))))
```

Result:

```
{ (Mother Bob Pam), (Mother Liz Pam) }
```

13.4 Long query form

```
!(morkql:q &family
  (quote
    (query
      (where
        (child $child $parent)
        (female $child))
      (yield
        (Daughter $child $parent))))))
```

This is exactly equivalent to a `find` with the same template and goals.

13.5 Repeated-variable constraint

```
!(morkql:q &pairs
  (quote
    (find (Reflexive $x)
      (related $x $x))))
```

The repeated variable requires both fields of `related` to be equal. No separate equality goal is needed.

13.6 Nested atoms

Assume MORK stores structured paper facts through the default structural codec:

```
(Paper
  (id p1)
  (references (Paper (id p2))))
```

A structural query may be written directly:

```
!(morkql:q &papers
  (quote
    (find (citation $paper $cited)
      (Paper
        (id $paper)
        (references
          (Paper (id $cited))))))))
```

The frontend treats the nested expression as a pattern AST and delegates physical encoding to the configured MORK codec.

13.7 Safe negation

```
!(morkql:q &jobs
  (quote
    (find (Available $person)
      (candidate $person)
      (not (blocked $person))))))
```

The query is safe because `$person` is bound by `candidate`. The following query is rejected:

```
(find (Available $person)
      (not (blocked $person)))
```

13.8 Grounded test

With a declared pure predicate `less-than`:

```
!(morkql:q &people
  (quote
    (find (Minor $person)
          (age $person $age)
          (test less-than $age 18))))
```

The compiler schedules the test after `$age` is bound.

13.9 Transitive closure

Assume `&graph` contains:

```
(edge a b)
(edge b c)
(edge c d)
```

Query:

```
!(morkql:collect
  (morkql:q &graph
    (quote
      (rules
        (export path)

        (<- (path $x $y)
            (edge $x $y))

        (<- (path $x $z)
            (path $x $y)
            (edge $y $z))))))
```

Result:

```
{ (path a b), (path a c), (path a d),
  (path b c), (path b d),
  (path c d) }
```

This rule group is positive, monotone, and finite range. It compiles to a MORKL fixed point whose body is a union of two MQT branches.

13.10 A helper relation and exports

```
(rules
  (export grandparent)

  (<- (two-hop $x $z)
```

```

    (parent $x $y)
    (parent $y $z))

(<- (grandparent $x $z)
    (two-hop $x $z)))

```

The fixed point may contain `two-hop` facts, but the result space contains only `grandparent` facts.

13.11 Schema-guided index selection

Given:

```

(relation parent
  (fields
    (parent Person)
    (child Person))
  (layout canonical
    (parent $parent $child))
  (index by-child
    (parent-by-child $child $parent)
    (maintained true)))

```

and the query:

```

(find (ParentOf Ann $parent)
  (parent $parent Ann))

```

an explain result may be:

```

(QueryPlan
  (logical
    (lookup parent
      (parent $parent Ann)))

  (physical
    (lookup-index parent by-child
      (parent-by-child Ann $parent)))

  (bindings
    (before Ann)
    (after Ann $parent))

  (proof
    (index-coherent parent by-child)))

```

13.12 Inspecting emitted MORKL

```

!(morkql:compile &family
  (quote
    (find (Mother $child $mother)
      (child $child $mother)
      (female $mother))))

```

A compiler may print either a normalized Core term or the native MORKL AST. The printed term must contain no `find`, `where`, `yield`, or `rules` constructors.

13.13 Proof-carrying compilation

```
!(morkql:compile/proof &graph
  (quote
    (rules
      (export path)
      (<- (path $x $y) (edge $x $y))
      (<- (path $x $z) (path $x $y) (edge $y $z))))))
```

A Verified implementation returns a compiled term together with checks for type safety, range restriction, positive recursion, finite range, rule lowering, fixed- point correctness, index assumptions, and any subsequent optimizer laws.

13.14 Pure query followed by explicit update

```
!(bind! &new-mothers
  (morkql:q &family
    (quote
      (find (Mother $child $mother)
        (child $child $mother)
        (female $mother)))))

; Querying did not mutate &family.
!(morkql:insert-space! &derived &new-mothers)
```

The effect is explicit in `insert-space!`; the original query remains a reusable virtual view.

14 Conformance

14.1 Base implementation requirements

A Base MORKQL 0.1 implementation must:

- parse the canonical ASCII surface syntax;
- implement `find`, the long query form, `union`, `intersect`, `except`, and positive `rules`;
- enforce variable range restriction and relation arity;
- compile all accepted queries to ordinary MORKL;
- implement the mandatory Core constructors supported by its declared MORKL target;
- provide `morkql:q`, `morkql:collect`, `morkql:explain`, and `morkql:compile`; and
- document its codec, schema mode, target version, and result-order policy.

14.2 Verified implementation requirements

A Verified implementation must additionally:

- use strict schemas for relation queries;
- check types, safe negation, stratification, monotonicity, and finite range;
- classify and validate every selected index;
- reject or explicitly declassify opaque grounded operations;

- emit a certificate for `compile/proof`; and
- provide an independently invocable certificate checker.

14.3 Determinism

Given the same normalized query, schema, target version, planner configuration, and source-independent statistics, a compiler should emit the same normalized plan and hashes. Runtime result order may vary unless canonical collection is requested.

14.4 Forward compatibility

Unknown surface forms must be rejected rather than interpreted as relation patterns when they occur in query-control position. Certificates must include language and target versions. New optional forms should use namespaced symbols until standardized.

15 Open Design Questions

The following details are intentionally left open for implementation experiments:

1. whether the host operation can receive query atoms non-eagerly, eliminating explicit `quote`;
2. the canonical serialization of Core paths under the evolving MORK codec;
3. the proof-term format used to connect frontend certificates to the Zippy prover artifacts;
4. the standard schema registry and index-maintenance protocol;
5. whether aggregates belong in the surface language or only in Core and grounded declarations;
6. whether parameterized named views should be surface rules, host functions, or compiled routines; and
7. which cost statistics can be exposed without becoming part of semantic conformance.

These questions do not affect the central semantic contract: a MORKQL query denotes a finite set of logical results and compiles to an equivalent MORKL path-space program.

A Compact Grammar Reference

```

QUERY :=
  (find TEMPLATE GOAL*)
  | (query (where GOAL*) (yield TEMPLATE))
  | (union QUERY QUERY+)
  | (intersect QUERY QUERY+)
  | (except QUERY QUERY)
  | (rules RULE-OPTION* CLAUSE+)
  | (core CORE-SPACE)

RULE-OPTION :=
  (export RELATION+)

CLAUSE :=
  (<- RELATION-PATTERN GOAL*)

GOAL :=
  RELATION-PATTERN
  | (not RELATION-PATTERN)

```

```

| (= TERM TERM)
| (!= TERM TERM)
| (test GROUNDED-NAME TERM*)

CORE-PATH :=
  (path-ref NAME)
| (path-lit ITEM*)
| (path-cat CORE-PATH+)
| (path-ground-pp NAME CORE-PATH)
| (path-ground-sp NAME CORE-SPACE)

CORE-SPACE :=
  (empty)
| (space NAME)
| (one CORE-PATH)
| (literal CORE-PATH*)
| (union CORE-SPACE CORE-SPACE+)
| (intersect CORE-SPACE CORE-SPACE+)
| (except CORE-SPACE CORE-SPACE)
| (under CORE-SPACE CORE-SPACE)
| (outside CORE-SPACE CORE-SPACE)
| (concat-spaces CORE-SPACE CORE-SPACE)
| (each-head CORE-SPACE NAME NAME CORE-SPACE)
| (fix CORE-SPACE NAME CORE-SPACE)
| (fold-heads CORE-SPACE CORE-PATH NAME NAME NAME
              CORE-SPACE CORE-PATH)
| (add-prefix CORE-PATH CORE-SPACE)
| (remove-prefix CORE-SPACE CORE-PATH)
| (tails CORE-SPACE)
| (common-tails CORE-SPACE)
| (ground-ps NAME CORE-PATH)
| (ground-ss NAME CORE-SPACE)
| (range CORE-SPACE INTEGER INTEGER)
| (call NAME (CORE-PATH*) (CORE-SPACE*))

```

B Compiler Correctness Skeleton

A mechanized frontend proof can be organized around the following lemmas.

Lemma 1: alpha-renaming

If $Q' \equiv_\alpha Q$, then $\llbracket Q' \rrbracket_D = \llbracket Q \rrbracket_D$.

Lemma 2: MQT induction

For any finite list of positive patterns $\vec{P}$ and template T , the expansion of $\text{MQT}(D, \vec{P}, T)$ equals the set of consistently instantiated templates. The proof is by induction on $\vec{P}$.

Lemma 3: conjunction permutation

For pure positive goals, any permutation preserves the satisfying substitution set. Dependency-respecting movement of tests and negation preserves the same result.

Lemma 4: layout substitution

If layout L is coherent for relation R , replacing a logical R lookup by L and applying its inverse field mapping preserves matching substitutions.

Lemma 5: anti-join

For safe negation, subtracting binding rows that extend to a match of the negated pattern yields exactly the substitutions satisfying logical negation.

Lemma 6: rule consequence

The union of clause-level MQT expansions denotes the immediate-consequence operator $T_{P,D}$.

Lemma 7: fixed point

If $T_{P,D}$ is monotone and finite range, inflationary iteration from I_0 terminates at $\text{lf}p\ I$. $I_0 \cup T_{P,D}(I)$.

Theorem: frontend correctness

Combining Lemmas 1–7 with structural induction over set-query forms gives:

$$\llbracket Q \rrbracket_{D,S} = \llbracket \text{compile}(Q, S) \rrbracket_{\text{path}(D,S)}.$$

This theorem ends at the initial MORKL term. Certified MORKL rewrite and executor equivalence results may be composed afterward.

C Recommended Explain-Plan Schema

```
(QueryPlan
  (version "0.1-draft")

  (logical
    (query-id "...")
    (output (Mother $child $mother))
    (goals
      (child $child $mother)
      (female $mother)))

  (static
    (types
      ($child Person)
      ($mother Person))
    (safe true)
    (stratum 0))

  (physical
    (step 0
      (relation child)
      (layout canonical)
      (bound-before)
      (bound-after $child $mother))
```



```

    (ground-prefix-length 1))

  (step 1
    (relation female)
    (layout canonical)
    (bound-before $child $mother)
    (bound-after $child $mother)
    (ground-prefix-length 2)))

(lowering
  (macro MQT)
  (target-hash "..."))

(proof
  (layout-evidence ...)
  (permutation ...)
  (lowering-theorem mqt-theorem)))

```

Implementations may add cost estimates, trie-navigation details, operation-graph node counts, or optimizer traces, but the logical and physical views should remain separate.

D Glossary

Atom. A logical METTA value or expression used as a fact, pattern, or result.

Binding row. An internal encoded substitution used while compiling joins and anti-joins.

Codec. The injective mapping between supported logical atoms and MORK paths.

Coherent index. A physical layout representing exactly the same logical fact set as the canonical layout.

Core. The lossless MORKQL syntax that maps one-to-one to MORKL constructors.

Layout. A physical path template for a logical relation.

MQT. The compiler macro that matches multiple patterns with consistent bindings and emits a template.

Path. A finite word over MORK path items.

Space. A finite set of paths, stored or virtual.

Stratum. A layer of a rule program evaluated after all negatively referenced lower layers.

Virtual view. A result space navigated through MORKL or zipper operations without mandatory eager materialization.

References

- [1] Adam Vandervorst. *Zippy: A practical algebra for sets of paths*, branch b. <https://github.com/Adam-Vandervorst/Zippy/tree/b>, accessed July 12, 2026.
- [2] Adam Vandervorst. *Zippy: An Algebra of Path Sets*. <https://github.com/Adam-Vandervorst/Zippy/blob/b/ALGEBRA.md>, accessed July 12, 2026.
- [3] Adam Vandervorst. *MORKL.scala*. <https://github.com/Adam-Vandervorst/Zippy/blob/b/src/main/scala/MORKL.scala>, accessed July 12, 2026.
- [4] Adam Vandervorst. *Zipper specification in Egglog*. <https://github.com/Adam-Vandervorst/Zippy/blob/b/zipper-spec.egg>, accessed July 12, 2026.

-
- [5] The MORK project. *Data in MORK*. <https://github.com/trueagi-io/MORK/wiki/Data-in-MORK>, accessed July 12, 2026.
 - [6] The MORK project. *Minimal MeTTa 2 (MM2)*. [https://github.com/trueagi-io/MORK/wiki/Minimal-MeTTa-2-\(MM2\)](https://github.com/trueagi-io/MORK/wiki/Minimal-MeTTa-2-(MM2)), accessed July 12, 2026.
 - [7] The OpenCog Hyperon project. *hyperon-experimental: MeTTa programming language implementation*. <https://github.com/trueagi-io/hyperon-experimental>, accessed July 12, 2026.
 - [8] The MeTTa project. *Main concepts*. https://metta-lang.dev/docs/learn/tutorials/eval_intro/main_concepts.html, accessed July 12, 2026.